

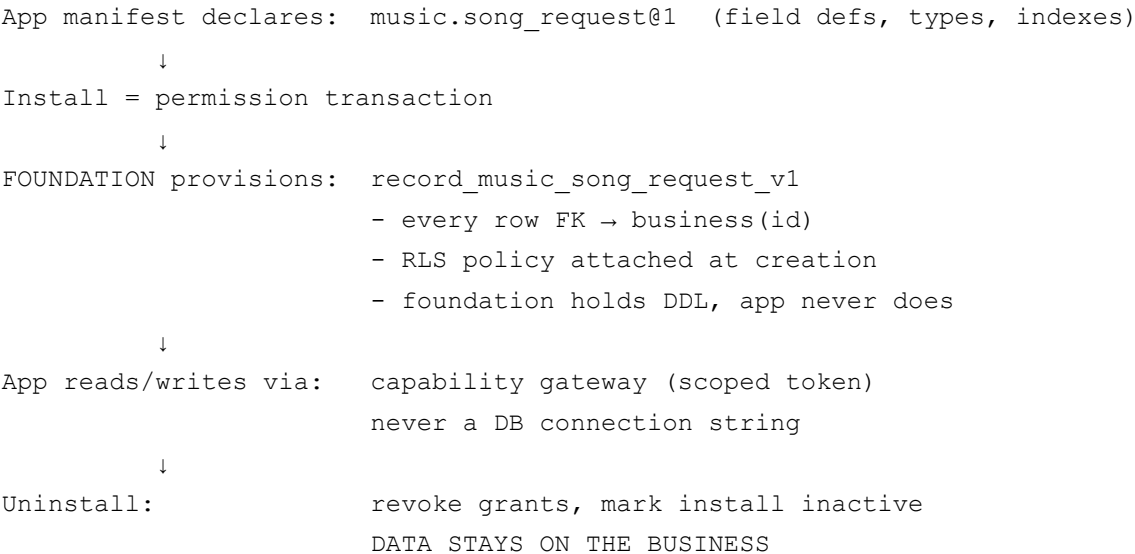
Ghost Foundation — What to Build, Front to Back

No apps. This is the platform kernel that apps get built on and tested against.

Design rule: One business = one entity. All data attaches to it. Any app, any tool, any employee writes to that same entity. Apps declare schemas; the foundation owns storage.

The structural mechanism (read this first)

This is what makes “apps don’t own data” actually work in a database.



Three storage tiers:

Tier	What	Who designs it
Universal core	business, person, membership, location, customer, product, availability, booking, media	You, by hand
Industry schemas	restaurant.menu, charter.trip, lodging.unit, music.performance	You, by hand, versioned
App schemas	anything an app declares	App declares, foundation provisions

All three sit under the same provenance layer and the same permission layer.

BACKEND MODULES

F1 · Identity kernel

Tables: `business`, `person`, `credential`, `membership`, `ownership_claim` Build: URN minting, signup, workspace switch, invite, ownership verification. **Done when:** three people with different roles act on one business and every action attributes to the right person.

F2 · Permission kernel

Tables: `permission`, `grant_record` Build: permission catalog seed, grant/revoke, effective-permission resolver with constraint evaluation. **Done when:** roles are pure shorthand — deleting all roles and using raw grants changes nothing about behavior.

F3 · Schema registry + provisioner ← *the keystone*

Tables: `schema_def`, `schema_provision` Build: JSON-schema → DDL compiler, table provisioner, RLS attacher, migration runner for version bumps, uninstall-safe teardown. **Done when:** you register a schema from a manifest, the table appears with correct types/indexes/FK/RLS, and the app can only reach it through the gateway.

F4 · Provenance engine ← *the differentiator*

Tables: `source`, `observation`, `fact`, `conflict` Build: observation ingest (append-only), 9-tier authority resolver, conflict detector, owner override with `owner_locked`, freshness decay. Authority order: owner → operational system → live integration → verified employee → transaction evidence → structured public → website → third party → AI inference. **Done when:** two sources disagree on Friday hours, a conflict row appears, the owner decides, and no inference can ever overwrite that decision.

F5 · Capability registry + router

Tables: `capability`, `provider`, `capability_impl` Build: capability catalog, typed I/O validation, provider resolution by priority + health, fallback chain. **Done when:** one capability call resolves to a provider and the caller cannot tell which one ran.

F6 · Policy gate

Build: deterministic evaluator → AUTO / ASK / NEVER + constraints. Internal function first; keep `PolicyProvider` interface for OPA. **Hard rule:** AI never participates in the decision.

Structured facts and cryptographic identity only. Natural-language persuasion is not a permission. **Done when:** no privileged capability can execute without passing the gate — proven by test, not convention.

F7 · Execution runtime

Tables: `execution`, `execution_step`, `approval` Build: queue, idempotency enforcement, retry with resume-vs-repeat semantics, approval suspend/resume from durable state, unknown-state halt. **Done when:** killing the worker mid-execution loses nothing and duplicates nothing.

F8 · Verification + ledger

Tables: `verification`, `evidence`, `ledger` Build: expected-state contracts per capability, verification readers, expected-vs-observed compare, graded result (`verified/partial/unknown/failed`), hash-chained append-only ledger. **Done when:** “the API returned 200” and “the destination now says what we asked” are separately queryable facts.

F9 · Event bus

Tables: `event_outbox`, `subscription` Build: transactional outbox, dispatcher, subscriber registry, dead-letter. Postgres LISTEN/NOTIFY first; NATS behind the interface later. **Done when:** state change and event insert commit in one transaction; zero events lost on dispatcher crash.

F10 · Connections + secrets

Tables: `connection`, `secret_ref` Build: OAuth/API-key/email/iCal/browser/device methods, health checks, reauth flow, envelope encryption. Apps get opaque refs only. **Done when:** no app, surface, or executor can obtain a plaintext credential.

F11 · App framework

Tables: `app`, `app_version`, `app_install` Build: manifest parser/validator, install as permission transaction, schema provisioning trigger, version upgrade with migration contract, uninstall that preserves data. **Done when:** install → uninstall → reinstall and the business data is intact and reattaches.

F12 · Control plane API

Build: single entry for dashboard, MCP, SMS, voice, apps, schedules. Correlation IDs, versioning, rate limits.

F13 • MCP gateway

Build: `/mcp/business/{business_id}` logical route over one shared implementation.

Resources projected from `fact` + provisioned records, filtered by `visibility_rule`. Tools = capabilities filtered by grant. **Done when:** an external AI client reads approved business data and invokes an approved capability, and every call lands in the ledger.

F14 • Realtime channel

Build: per-business subscription, event fan-out to connected clients.

F15 • Automation engine

Tables: `automation`, `automation_run` Build: WHEN → IF → THEN → VERIFY. Steps are capability calls; approval gates are first-class. **Done when:** an automation with an approval gate suspends, texts the owner, and resumes days later from durable state.

FRONTEND MODULES

U1 • Component library

The ten primitives every surface composes from: Approval Gate · Availability Card · Business Hours · Customer Match · Permissioned Message Send · Public Profile Block · Retry + Idempotency · Review Request Step · Source Reconciliation · Verification Step

U2 • Block renderer

15 block types: card, accordion, form, table, calendar, gallery, list, chart, map, modal, drawer, button, search, status/badge, media player. Blocks bind to `field_path` or `capability_key` — never to an app's private store.

U3 • Dashboard shell

Home / Business / Apps / Connections / Automations / Ghost / Devices. Linktree-style: tap to expand inline, tap to collapse, open full workspace when needed. Mobile-first.

U4 • Data browser ← *makes the thesis visible*

Every record inspectable independently of any app. Each shows: source · last update · verification state · which apps have access. **Done when:** the owner can see their data with every app uninstalled.

U5 • Permission UI

Plain-language grants. "ABC Marketing can update specials but cannot read customer phone numbers." Advanced view exposes generated policy.

U6 · Conflict resolver UI

Source-vs-source diffs, confidence, provenance, owner decision. This is F4's face.

U7 · Execution/receipt UI

Task list, live state, approval inbox, receipt with expected-vs-observed and evidence.

U8 · Surface builder

Audience → capabilities → layout → theme → publish targets → generated scoped policy.

U9 · Setup flow

Not 17 steps. Claim → verify → discover → confirm facts → one working thing. Everything else earned progressively after value lands.

U10 · System area

Connections health, devices, AI routing, policy inspector, export/import.

BUILD ORDER

Sequential — nothing parallelizes around these:

F1 → F2 → F3 → F4 → F5 → F6 → F9

F3 before F4 because provenance covers provisioned tables too. F6 before anything executes.

Then three parallel tracks:

- **A (spine):** F7 → F8 → F15
- **B (interface):** F12 → F14 → F13
- **C (surface):** U1 → U2 → U3 → U4 → U6 → U7

Then: F10 → F11 → U5 → U8 → U9 → U10

Executors last, in this order: API → browser → device endpoint → Android. Android carries

platform-ToS and account-ban risk to your customers' real business pages; it is not the demo.

FOUNDATION ACCEPTANCE TEST

The foundation is done when you can build three throwaway apps against it that share business identity, permissions, events, and dashboard with zero pairwise integration code — and then:

1. Replace one app → other apps and data keep working
2. Replace the surface → structured data unchanged
3. Replace the model → capabilities and workflows unchanged
4. Move runtime cloud → mini-PC → identity, grants, schemas, workflows, data survive
5. Remove one app → it loses capability access without taking data with it

If replacing a component requires rebuilding the business, that component is still too tightly coupled.

STILL OPEN

Customer identity scope. You said one business = one entity, all data attaches to it. Clear for the business side. Not yet settled: when a tourist interacts with four different businesses, is that one `person` across all four, or four separate customer records?

- **One person** → cross-business recommendations, portable identity, GCR consumer layer, loyalty that follows the customer. Also: real privacy surface area and consent design.
- **Per-business** → simpler, safer, no cross-business inference. But no portable customer identity.

The kernel above supports either — `person` exists independently and `membership` is the business link. Customer records would either reference `person` or live per-business. This decision changes F4, F13, and the entire consumer/GCR layer, so it needs settling before F4 ships.